

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)

Assessment Tool Weightage: 35%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5
8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5

9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

*I **Kadhirezhilan. D [192511228]** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q1. Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.

Answer

Problem Understanding

A DBMS converts an SQL query into an executable form and produces the required result.

The query processing pipeline consists of:

1. SQL input
2. Parsing and validation
3. Query tree / relational algebra generation
4. Query optimization
5. Execution plan generation
6. Query execution
7. Result output

Pseudocode

ALGORITHM Query_Processing_Pipeline

INPUT:

SQL_Query

OUTPUT:

Query_Result

BEGIN

// Step 1: Receive SQL query

READ SQL_Query

IF SQL_Query is empty THEN

 DISPLAY "Invalid: Query is empty"

 RETURN

END IF

// Step 2: Lexical and syntax analysis

Tokens $\leftarrow$ Tokenize(SQL_Query)

IF Tokens are invalid THEN

 DISPLAY "Syntax Error"

 RETURN

END IF

// Step 3: Parse the SQL query

Parse_Tree $\leftarrow$ Parse(Tokens)

IF Parse_Tree is invalid THEN

 DISPLAY "Parsing Error"

 RETURN

END IF

// Step 4: Semantic validation

IF NOT Validate_Tables(Parse_Tree) THEN

 DISPLAY "Table does not exist"

 RETURN

END IF

IF NOT Validate_Attributes(Parse_Tree) THEN

 DISPLAY "Invalid attribute"

 RETURN

END IF

// Step 5: Convert query into relational algebra

Relational_Expression $\leftarrow$

 Convert_To_Relational_Algebra(Parse_Tree)

```
// Step 6: Optimize query
Optimized_Expression ←
    Optimize(Relational_Expression)
```

```
// Optimization may include:
// Selection pushdown
// Projection pushdown
// Efficient join ordering
// Join algorithm selection
```

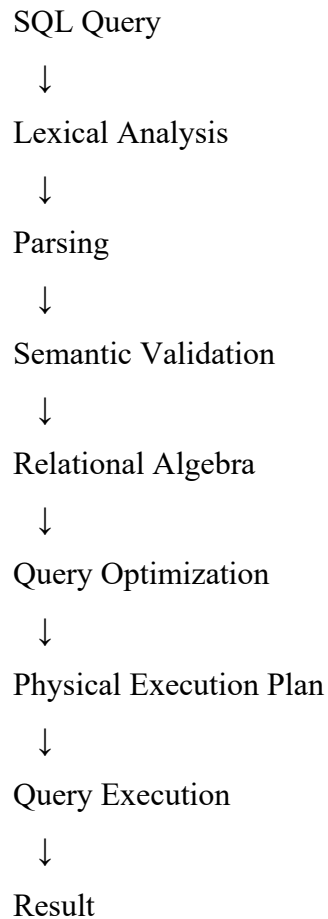
```
// Step 7: Generate physical execution plan
Execution_Plan ←
    Generate_Execution_Plan(Optimized_Expression)
```

```
// Step 8: Execute the plan
Query_Result ←
    Execute(Execution_Plan)
```

```
// Step 9: Return result
DISPLAY Query_Result
```

END

Processing Flow



Explanation

- **Parsing** checks the syntax of the SQL statement.
- **Semantic validation** checks whether the referenced tables and attributes exist.
- **Optimization** selects an efficient execution strategy.
- **Execution** performs the actual operations such as selection, projection, and joins.

Conclusion

The complete query processing pipeline transforms a high-level SQL query into an optimized execution plan and executes it to produce the final result.

Q2. Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

Answer

Problem Understanding

Cost-based query optimization evaluates different execution plans and selects the plan with the **lowest estimated cost**.

For join optimization, the optimizer estimates the number of disk I/O operations required for different join orders.

Inputs

- Relations
- Number of pages in each relation
- Number of tuples
- Available buffer pages
- Possible join orders

Output

- Lowest-cost join order
- Estimated minimum I/O cost

Pseudocode

ALGORITHM Cost_Based_Join_Optimization

INPUT:

Relations R

Page_Count[R]

Tuple_Count[R]

Buffer_Pages

Join_Conditions

OUTPUT:

Best_Join_Order

Minimum_Cost

BEGIN

Minimum_Cost $\leftarrow$ INFINITY

Best_Join_Order $\leftarrow$ NULL

// Generate all possible join orders

Join_Orders $\leftarrow$ Generate_Join_Orders(R)

FOR each Order IN Join_Orders DO

Current_Cost $\leftarrow$ 0

Intermediate_Size $\leftarrow$ NULL

FOR each Join IN Order DO

Left_Relation $\leftarrow$ Join.Left

Right_Relation $\leftarrow$ Join.Right

// Estimate cost of the join

Join_Cost $\leftarrow$

Estimate_Join_IO_Cost(

Left_Relation,

Right_Relation,

Buffer_Pages

)


```
Current_Cost ←  
    Current_Cost + Join_Cost
```

```
// Estimate size of intermediate result  
Intermediate_Size ←  
    Estimate_Join_Result_Size(  
        Left_Relation,  
        Right_Relation  
    )
```

```
// Treat result as relation for next join  
Update_Intermediate_Relation(  
    Intermediate_Size  
)
```

```
END FOR
```

```
// Compare current plan with best plan  
IF Current_Cost < Minimum_Cost THEN
```

```
    Minimum_Cost ← Current_Cost  
    Best_Join_Order ← Order
```

```
END IF
```

```
END FOR
```

```
RETURN Best_Join_Order, Minimum_Cost
```

END

Cost Estimation Example

For a simple page-oriented nested-loop join:

$$\text{Cost} = B(R) + B(R) \times B(S)$$

where:

- $B(R)$ = pages of R
- $B(S)$ = pages of S

For example:

$$B(R) = 100, B(S) = 1000$$

Then:

$$\begin{aligned}\text{Cost} &= 100 + (100 \times 1000) \\ &= 100100\end{aligned}$$

The optimizer compares this with other possible plans.

Important Optimization Principle

If:

$$R \bowtie S$$

produces a small intermediate result, that result can be joined with another relation before performing a large join.

Therefore:

Choose the join order with minimum estimated I/O cost

Conclusion

The cost-based optimizer generates possible join orders, estimates their I/O costs, compares them, and selects the least-cost execution plan.

Q3. Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

Answer

Problem Understanding

Nested Loop Join compares tuples from two relations.

Let:

- Relation R contain tuples.
- Relation S contain tuples.

For every tuple in R, the algorithm checks tuples in S.

Pseudocode

ALGORITHM Nested_Loop_Join

INPUT:

Relation R

Relation S

Join_Condition

OUTPUT:

Joined_Relation

BEGIN

Result $\leftarrow$ Empty

FOR each tuple r IN R DO

FOR each tuple s IN S DO

IF Join_Condition(r, s) = TRUE THEN

Add (r, s) TO Result

END IF

END FOR

END FOR

RETURN Result

END

Block/Page-Based Nested Loop Join

A more efficient version uses buffer pages.

Let:

- = number of pages in R
- = number of pages in S
- = number of available buffer pages

If R is the outer relation, the block nested-loop join cost can be approximated by:

$$B(R) + \lceil B(R) / M - 2 \rceil B(S)$$

Pseudocode

ALGORITHM Block_Nested_Loop_Join

INPUT:

Relation R

Relation S

M = Number of Buffer Pages

OUTPUT:

Result

BEGIN

Result $\leftarrow$ Empty

```
FOR each block BR of R
  containing at most  $(M - 2)$  pages DO

  Load BR into memory

  FOR each block BS of S DO

    Load BS into memory

    FOR each tuple r IN BR DO

      FOR each tuple s IN BS DO

        IF Join_Condition(r, s) THEN
          Add (r, s) TO Result
        END IF

      END FOR

    END FOR

  END FOR

END FOR

RETURN Result

END
```

Time Complexity

At the tuple level:

$$O(n \times m)$$

where:

- n = tuples in R
- m = tuples in S

Therefore, if:

$$n=1000, m=10000$$

the worst-case number of tuple comparisons is:

$$\begin{aligned} &1000 \times 10000 \\ &= 10,000,000 \end{aligned}$$

So approximately **10 million comparisons** may be required.

I/O Complexity

Using buffer pages:

$$B(R) + \lceil B(R) / M - 2 \rceil B(S)$$

The choice of outer relation and the number of available buffers significantly affect performance.

Conclusion

Nested Loop Join is simple and works for general join conditions, but it can be expensive for large relations because many tuple or page comparisons may be required.

Q4. Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.

Answer

Problem Understanding

A deadlock occurs when transactions wait for one another indefinitely.

A **Wait-For Graph (WFG)** represents waiting relationships.

- Each transaction is represented as a node.
- An edge means T_i is waiting for T_j .

Deadlock Condition

Cycle in WFG=Deadlock

Pseudocode

ALGORITHM Detect_Deadlock

INPUT:

Transactions

Wait_For_Graph

OUTPUT:

Deadlock_Status

BEGIN

FOR each transaction T DO

Visited[T] ← FALSE

RecursionStack[T] ← FALSE

END FOR

FOR each transaction T DO

IF Visited[T] = FALSE THEN

IF DFS_Cycle_Detection(

T,

Visited,

RecursionStack

) = TRUE THEN

DISPLAY "Deadlock Detected"

RETURN TRUE

END IF

END IF

END FOR

DISPLAY "No Deadlock"

RETURN FALSE

END

DFS Cycle Detection

```
FUNCTION DFS_Cycle_Detection(  
    T,  
    Visited,  
    RecursionStack  
)
```

BEGIN

Visited[T] ← TRUE

RecursionStack[T] ← TRUE

FOR each transaction U
connected from T DO

IF Visited[U] = FALSE THEN

IF DFS_Cycle_Detection(
 U,


```
        Visited,  
        RecursionStack  
    ) = TRUE THEN  
  
        RETURN TRUE  
  
    END IF  
  
    ELSE IF RecursionStack[U] = TRUE THEN  
  
        RETURN TRUE  
  
    END IF  
  
END FOR  
  
RecursionStack[T] ← FALSE  
  
RETURN FALSE  
  
END
```

Example

Suppose:

$T1 \rightarrow T2$

$T2 \rightarrow T3$

$T3 \rightarrow T1$

There is a cycle:

$T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$

Therefore:

Deadlock Detected

If the graph is:

$T1 \rightarrow T2 \rightarrow T3$

there is no cycle.

Therefore:

No Deadlock

Conclusion

The Wait-For Graph method detects deadlocks by checking whether the graph contains a cycle. DFS can efficiently detect such cycles.

Q5. Design pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.

Answer

Problem Understanding

Basic Two-Phase Locking (2PL) ensures conflict serializability by dividing transaction execution into two phases:

Growing Phase

A transaction can acquire locks but cannot release them.

Shrinking Phase

A transaction can release locks but cannot acquire new locks.

Lock Types

Shared Lock (S)

Used for reading.

Exclusive Lock (X)

Used for writing.

Pseudocode

ALGORITHM Basic_2PL(Transaction T)

INPUT:

Transaction T

Operations of T

OUTPUT:

Completed Transaction

BEGIN

Phase $\leftarrow$ GROWING

FOR each operation O in T DO

IF O is READ(X) THEN

IF Phase = SHRINKING THEN

DISPLAY "Lock acquisition not allowed"

WAIT or ABORT

END IF

Request_Shared_Lock(T, X)

IF Lock_Granted THEN

READ(X)

ELSE

WAIT

END IF

ELSE IF O is WRITE(X) THEN

IF Phase = SHRINKING THEN

DISPLAY "Lock acquisition not allowed"

WAIT or ABORT

END IF

Request_Exclusive_Lock(T, X)

IF Lock_Granted THEN

WRITE(X)

ELSE

WAIT

END IF

END IF

END FOR

Phase $\leftarrow$ SHRINKING

FOR each Lock held by T DO

Release_Lock(T, Lock)

END FOR

COMMIT T

END

Lock Request Procedure

FUNCTION Request_Shared_Lock(T, X)

BEGIN

IF X is not locked exclusively THEN

Grant S-Lock to T

RETURN TRUE

ELSE

Add T to Waiting_Queue(X)

RETURN FALSE

END IF

END

Exclusive Lock Request

FUNCTION Request_Exclusive_Lock(T, X)

BEGIN

IF X has no conflicting lock THEN

Grant X-Lock to T

RETURN TRUE

ELSE

Add T to Waiting_Queue(X)

RETURN FALSE

END IF

END

Release Procedure

FUNCTION Release_Lock(T, X)

BEGIN

Remove T's lock on X

IF another transaction is waiting THEN

Check waiting transactions

Grant compatible lock

END IF

END

Important Rule

Once the transaction starts releasing locks:

It cannot acquire any new lock

Conclusion

Basic 2PL guarantees conflict serializability by separating lock acquisition and lock release into growing and shrinking phases. However, basic 2PL can still result in deadlocks.

Q6. Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.

Answer

Problem Understanding

Timestamp ordering assigns every transaction a unique timestamp.

For each data item X, the DBMS maintains:

- **Read_TS(X)** – largest timestamp of a transaction that successfully read X.
- **Write_TS(X)** – largest timestamp of a transaction that successfully wrote X.

Thomas' Write Rule modifies the standard timestamp-ordering protocol by allowing certain obsolete writes to be ignored instead of aborting the transaction.

Basic Rules

For a transaction with timestamp :

Read(X)

If:

$$TS(T) < Write_TS(X)$$

then T is attempting to read a value that has already been overwritten by a newer transaction.

Therefore, T is rejected/aborted.

Otherwise:

$$Read_TS(X) = \max(Read_TS(X), TS(T))$$

Write(X)

For Thomas' Write Rule:

If:

$$TS(T) < Read_TS(X)$$

then the write is obsolete and the transaction is aborted under the basic timestamp ordering approach.

If:

then the write is **obsolete**.

Thomas' Write Rule says:

$$TS(T) < Write_TS(X)$$

rather than aborting T.

Otherwise, perform the write and update:

Write_TS(X)=TS(T)

Pseudocode

ALGORITHM Thomas_Write_Rule

INPUT:

Transaction T

Operation O on data item X

TS(T)

Read_TS(X)

Write_TS(X)

OUTPUT:

Action

BEGIN

IF O = READ(X) THEN

IF TS(T) < Write_TS(X) THEN

DISPLAY "Read rejected"

ABORT T

RETURN

ELSE

Perform READ(X)

Read_TS(X) ←

MAX(Read_TS(X), TS(T))

RETURN "Read Successful"

END IF

ELSE IF O = WRITE(X) THEN

IF TS(T) < Read_TS(X) THEN

DISPLAY "Write rejected"

ABORT T

RETURN

ELSE IF TS(T) < Write_TS(X) THEN

DISPLAY "Obsolete write ignored"

RETURN "Write Ignored"

ELSE

Perform WRITE(X)

Write_TS(X) $\leftarrow$ TS(T)

RETURN "Write Successful"

END IF

END IF

END

Example

Suppose:

$TS(T1)=5$

and:

$Write_TS(X)=10$

T1 tries:

WRITE(X)

Since:

$5 < 10$

the write is obsolete.

Thomas' Write Rule says:

Ignore the write instead of aborting T1

Advantage

Thomas' Write Rule allows more schedules to proceed than basic timestamp ordering because obsolete writes can be ignored.

Conclusion

Thomas' Write Rule improves concurrency by ignoring obsolete write operations that cannot affect the final database state, thereby avoiding unnecessary transaction aborts.

Q7. Write a pseudocode algorithm for ARIES recovery with Analysis, Redo, and Undo phases.

Answer

Problem Understanding

ARIES stands for:

Algorithm for Recovery and Isolation Exploiting Semantics

Algorithm for Recovery and Isolation Exploiting Semantics

ARIES is a widely used database recovery technique based on **Write-Ahead Logging (WAL)**.

ARIES recovery consists of three major phases:

1. Analysis
2. Redo
3. Undo

Phase 1 – Analysis

The analysis phase determines:

- Which transactions were active at crash time.
- Which transactions committed.
- Which transactions were incomplete.
- The starting point for redo.

It reconstructs the transaction table and dirty page table.

Phase 2 – Redo

Redo repeats history from the appropriate point in the log.

The goal is to restore the database to the state that existed immediately before the crash.

Phase 3 – Undo

Undo rolls back the effects of transactions that were active when the crash occurred.

Pseudocode

ALGORITHM ARIES_Recovery

INPUT:

Log
Database
Checkpoint

OUTPUT:

Recovered Database

BEGIN

// -----

// PHASE 1: ANALYSIS

// -----

Read the latest valid checkpoint

Initialize Transaction_Table

Initialize Dirty_Page_Table

FOR each log record from checkpoint DO

IF record is START transaction THEN

 Add transaction to Transaction_Table

ELSE IF record is UPDATE THEN

 Update Transaction_Table

 Update Dirty_Page_Table

ELSE IF record is COMMIT THEN

 Mark transaction as COMMITTED

ELSE IF record is ABORT THEN

 Mark transaction as ABORTING

END IF

END FOR

Identify transactions active

at the time of crash

Determine Redo_Start_Point

// -----

// PHASE 2: REDO

// -----

FOR each log record starting from

Redo_Start_Point DO

IF record represents an update THEN

IF update is not already
reflected on disk THEN

REDO the update

END IF

END IF

END FOR

// -----

// PHASE 3: UNDO

// -----

FOR each transaction marked

ACTIVE or ABORTING DO

Follow its log records
backward

FOR each update record DO

 UNDO the update

 Write a compensation
 log record

END FOR

Write ABORT/END record

END FOR

RETURN Recovered Database

END

ARIES Flow

System Crash



Analysis



Find active transactions



Redo



Repeat required history



Undo



Rollback incomplete transactions



Recovered Database

Conclusion

ARIES provides reliable recovery by analyzing the log, redoing necessary history, and undoing incomplete transactions. Its three-phase structure ensures database consistency after system failure.

Q8. Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

Answer

Problem Understanding

Shadow Paging is a recovery technique that maintains two page tables:

1. **Shadow Page Table**
2. **Current Page Table**
 - The shadow page table points to the stable database pages.
 - The current page table is modified during transaction execution.
 - If the transaction commits, the current page table becomes the new stable page table.
 - If a failure occurs, the system discards the current page table and uses the shadow page table.

Pseudocode

ALGORITHM Shadow_Paging_Recovery

INPUT:

Database

Transaction_Operations

OUTPUT:

Recovered Database

BEGIN

// Step 1: Create shadow page table

Shadow_Page_Table $\leftarrow$

Copy(Current_Page_Table)

// Step 2: Create current page table

Current_Page_Table $\leftarrow$

Copy(Shadow_Page_Table)

// Step 3: Execute transaction

FOR each operation O DO

IF O modifies a page P THEN

New_Page $\leftarrow$ Copy(P)

Apply O to New_Page

Update Current_Page_Table
to point to New_Page

ELSE

Perform read operation

END IF

END FOR

// Step 4: Commit

IF Transaction_Committed THEN

Flush modified pages to disk

Shadow_Page_Table ←
Current_Page_Table

Commit transaction

DISPLAY "Transaction committed"

ELSE

// Step 5: Rollback

Current_Page_Table ←
Shadow_Page_Table

DISPLAY "Transaction rolled back"

END IF

END

Failure Handling

If a system crash occurs before commit:

Current Page Table



Discard



Shadow Page Table



Stable Database

Therefore:

Current Page Table → Discard

and:

Shadow Page Table → Restore

Advantages

- No traditional UNDO log is required.
- Recovery is simple.
- Stable pages remain protected.
- Rollback can be fast.
-

Conclusion

Shadow paging maintains a stable shadow copy and a modified current page table. On successful commit, the current table becomes permanent. On failure, the current table is discarded and the shadow table restores the previous consistent state.

Q9. Write pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.

Answer

Problem Understanding

In Immediate Update, database pages can be written before a transaction commits.

Therefore, after a crash:

- Committed transactions require **REDO**.
- Uncommitted transactions require **UNDO**.

Thus:

Immediate Update = UNDO + REDO

Log Format

Assume log records contain:

<T_i, X, Old_Value, New_Value>

Example:

<T₁, A, 100, 150>

means T₁ changed A from 100 to 150.

Pseudocode

ALGORITHM Immediate_Update_Recovery

INPUT:

Log

Database

OUTPUT:

Recovered Database

BEGIN

Committed $\leftarrow$ Empty_Set

Active $\leftarrow$ Empty_Set

// -----

// STEP 1: Scan the log

// -----

FOR each record R in Log DO

IF R is START(T) THEN

 Add T to Active

ELSE IF R is COMMIT(T) THEN

 Add T to Committed

 Remove T from Active

END IF

END FOR

// -----

// STEP 2: REDO

// -----

FOR each record R in Log from
beginning to end DO

IF R is UPDATE(T, X, Old, New)
AND T is in Committed THEN

Database[X] $\leftarrow$ New

END IF

END FOR

// -----

// STEP 3: UNDO

// -----

FOR each record R in Log from
end to beginning DO

IF R is UPDATE(T, X, Old, New)
AND T is in Active THEN

Database[X] $\leftarrow$ Old

END IF

END FOR

RETURN Database

END

Example

Suppose:

<START T1>

<T1, A, 100, 150>

<COMMIT T1>

<START T2>

<T2, B, 200, 250>

SYSTEM CRASH

Transaction Status

T1:

Committed

T2:

Uncommitted

REDO

For T1:

A=150

UNDO

For T2:

B=200

Final Database

A = 150

B = 200

Conclusion

Immediate Update recovery uses both UNDO and REDO operations to restore database consistency after a system crash.

Q10. Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

Answer

Problem Understanding

A database connectivity manager is responsible for managing database connections efficiently.

It should:

1. Create or maintain a connection pool.
2. Obtain a connection.
3. Execute SQL queries.
4. Commit successful transactions.
5. Roll back transactions when exceptions occur.
6. Return the connection to the pool.

Connection pooling avoids creating a new database connection for every request.

Pseudocode

ALGORITHM Database_Connectivity_Manager

INPUT:

SQL_Query

Query_Parameters

OUTPUT:

Query_Result

BEGIN

// Step 1: Initialize connection pool

IF Connection_Pool does not exist THEN

 Connection_Pool ←

 Create_Connection_Pool()

END IF

// Step 2: Obtain connection

Connection ←

 Connection_Pool.GetConnection()

TRY

// Step 3: Disable automatic commit

Connection.SetAutoCommit(FALSE)

// Step 4: Prepare SQL query

Statement ←

 Connection.PrepareStatement(SQL_Query)

// Step 5: Set parameters

FOR each parameter P DO

 Statement.SetParameter(P)

END FOR

```
// Step 6: Execute query  
IF SQL_Query is SELECT THEN
```

```
    Result ←  
        Statement.ExecuteQuery()
```

```
ELSE
```

```
    Result ←  
        Statement.ExecuteUpdate()
```

```
END IF
```

```
  
// Step 7: Commit transaction  
Connection.Commit()
```

```
  
// Step 8: Return result  
RETURN Result
```

```
  
CATCH DatabaseException E
```

```
    // Step 9: Rollback on error  
    Connection.Rollback()
```

```
    DISPLAY "Transaction rolled back"
```

```
    DISPLAY E.Message
```

```
    RETURN FAILURE
```


FINALLY

// Step 10: Close statement

IF Statement exists THEN

Statement.Close()

END IF

// Step 11: Return connection

IF Connection exists THEN

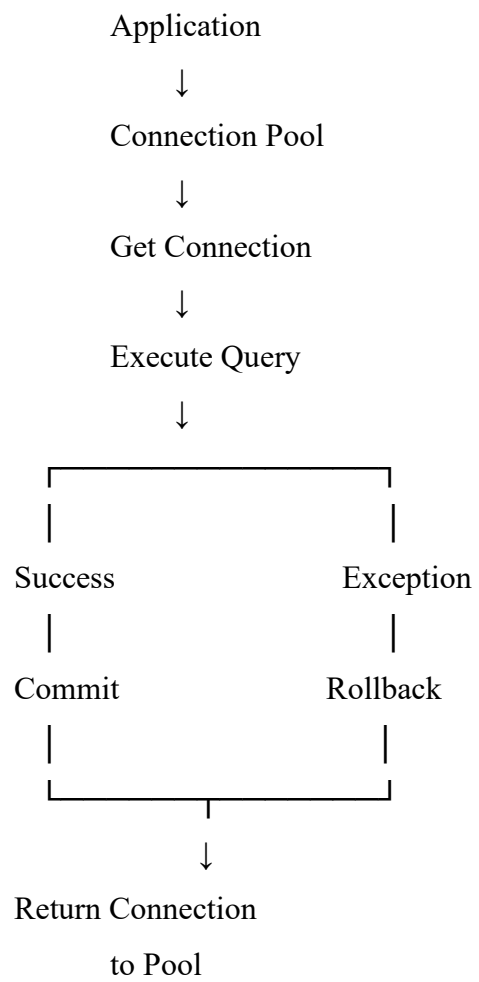
Connection_Pool.ReturnConnection(
Connection
)

END IF

END TRY

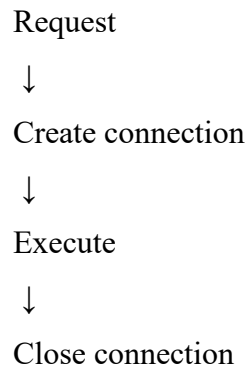
END

Connection Pooling Flow



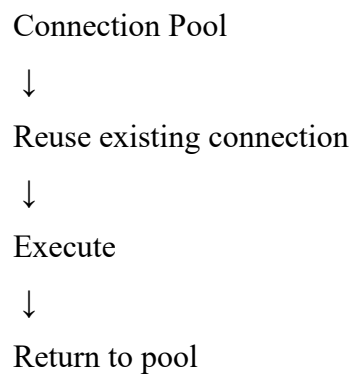
Why Connection Pooling?

Without pooling:



This repeated creation and closing is expensive.

With pooling:



This improves:

- Performance
- Resource utilization
- Response time
- Scalability

Exception Rollback

If an error occurs during a transaction:

```
TRY
    Execute operations
    COMMIT

CATCH
    ROLLBACK
```

FINALLY

Return connection

This prevents partially completed database transactions.

Conclusion

A database connectivity manager improves reliability by combining connection pooling, parameterized query execution, transaction commit, exception rollback, and proper resource management.